RodaFix: Sistema de Gestión para un Taller de Reparación de Bicicletas

Juan Verano Murcia

Nicolas Rodriguez Sanchez

Corporación Unificada Nacional de Educación Superior - CUN

Programación Orientada a Objetos I

Pedro Daniel Ospina Arias

29 de septiembre de 2026

RodaFix: Sistema de Gestión para un Taller de Reparación de Bicicletas

Planteamiento del Problema

En un taller de bicicletas de barrio, el dueño suele anotar en un cuaderno los datos de cada cliente, la bicicleta que trae y qué reparación pidió. Ese método funciona mientras el negocio es pequeño, pero se vuelve un problema cuando el taller crece: las hojas se pierden, dos personas no pueden consultar el mismo cuaderno al mismo tiempo y no hay forma rápida de saber cuántas órdenes están pendientes o cuánto se debe cobrar por una reparación ya hecha.

RodaFix busca resolver ese problema con un programa de consola escrito en Java que registre tres tipos de información relacionada entre sí: los clientes del taller, las bicicletas que cada cliente ha llevado a reparar y las órdenes de trabajo asociadas a cada bicicleta. La idea no es construir un sistema comercial completo, sino aplicar de forma práctica los conceptos de clases, objetos, encapsulamiento, herencia, polimorfismo y relaciones entre clases que corresponden al primer curso de Programación Orientada a Objetos.

Objetivo General

Desarrollar un prototipo funcional en Java, ejecutado por consola, que permita registrar clientes, sus bicicletas y las órdenes de reparación asociadas, usando arreglos nativos como estructura de almacenamiento y aplicando los principios básicos de la Programación Orientada a Objetos: clases, atributos privados, constructores, encapsulamiento (getters y setters), relaciones de agregación, composición y asociación entre clases, herencia y polimorfismo.

Objetivos Específicos

Diseñar las clases Cliente, Bicicleta y OrdenReparacion con sus atributos privados y sus respectivos constructores.

Implementar arreglos nativos con un atributo contador que controle cuántos elementos se han agregado y evite que el programa intente guardar más datos de los que caben en el arreglo.

Simular, mediante arreglos, la relación de composición entre un cliente y sus bicicletas, y la relación de asociación entre una bicicleta y sus órdenes de reparación.

Aplicar herencia y polimorfismo mediante las clases OrdenUrgente y OrdenMantenimiento, que extienden de OrdenReparacion y sobrescriben el método calcularCostoFinal().

Construir una clase Main que cree los objetos principales, llene los arreglos hasta su límite y muestre la información por consola de forma ordenada.

Modelado de Clases

El sistema se organiza en seis clases. Tres de ellas representan entidades del taller (Cliente, Bicicleta y OrdenReparacion), otras dos (OrdenUrgente y OrdenMantenimiento) son tipos especiales de orden que heredan de OrdenReparacion, y la sexta es la clase Taller, que actúa como contenedor general del negocio. A continuación se describe cada una y la relación que tiene con las demás.

Clase Cliente

Representa a la persona dueña de una o varias bicicletas. Sus atributos privados son cedula (String), nombre (String), telefono (String) y un arreglo de tipo Bicicleta[] llamado bicicletas, junto con un contador entero que lleva el registro de cuántas bicicletas tiene guardadas ese cliente. La capacidad del arreglo se define en el constructor, por ejemplo 5 bicicletas por cliente.

Cliente tiene el método agregarBicicleta(Bicicleta b), que primero revisa si el contador es menor que la capacidad del arreglo antes de guardar el objeto; si el arreglo ya está lleno, el método avisa por consola que no hay espacio disponible. También cuenta con el método mostrarInformacion(), que imprime los datos del cliente y recorre el arreglo de bicicletas para mostrar cada una.

La relación entre Cliente y Bicicleta es de composición: cada bicicleta pertenece a un cliente y no tiene sentido dentro del sistema si no está asociada a alguien. En el diagrama de clases esto se representa con un rombo relleno del lado de Cliente.

Clase Bicicleta

Representa cada bicicleta que un cliente lleva al taller. Sus atributos privados son codigo (String, un identificador interno del taller), marca (String), modelo (String), tipo (String, por ejemplo montaña, ruta o urbana) y un arreglo de tipo OrdenReparacion[] llamado historialOrdenes, con su respectivo contador para controlar cuántas órdenes tiene registradas esa bicicleta. Como el arreglo es de tipo OrdenReparacion, puede guardar tanto órdenes normales como órdenes urgentes o de mantenimiento.

Al igual que Cliente, Bicicleta tiene un método agregarOrden(OrdenReparacion o) que valida el contador contra la capacidad antes de insertar, y un método mostrarInformacion() que imprime los datos propios de la bicicleta y el detalle de cada orden guardada en su historial. Además tiene el método calcularTotal(), que recorre el historial y suma el costo final de cada orden; cada orden calcula el suyo según su tipo.

La relación entre Bicicleta y OrdenReparacion es de asociación: una orden de reparación existe para atender una bicicleta específica, pero conceptualmente podría documentarse aparte (por ejemplo, en un reporte de facturación) sin que eso implique que deje de existir si la bicicleta se elimina del sistema.

Clase OrdenReparacion

Representa un trabajo puntual realizado sobre una bicicleta: cambio de frenos, ajuste de cambios, mantenimiento general, entre otros. Sus atributos privados son numeroOrden (int), descripcionTrabajo (String), fecha (String), costo (double) y estado (String, con valores como "pendiente", "en proceso" o "entregado"). Esta clase no contiene arreglos porque su función es guardar el detalle de un trabajo ya realizado o por realizar.

OrdenReparacion cuenta con su constructor, sus getters y setters (el estado, por ejemplo, se puede actualizar con un setter cuando el trabajo pasa de "pendiente" a "entregado"), un método calcularCostoFinal(), que en la orden normal devuelve el mismo costo base, y un método mostrarInformacion() que imprime el detalle del trabajo por consola. Esta clase funciona como clase padre de OrdenUrgente y OrdenMantenimiento.

Clase OrdenUrgente

Hereda de OrdenReparacion mediante extends y representa un trabajo que el cliente pide con prioridad. Agrega un atributo privado recargo (double), que es un porcentaje, y en su constructor usa super() para inicializar los datos heredados. Sobrescribe con @Override el método calcularCostoFinal(), que suma el recargo al costo base (por ejemplo, un trabajo de $20.000 con recargo del 20% queda en $24.000), y sobrescribe mostrarInformacion() para indicar que la orden es urgente antes de mostrar el detalle heredado.

Clase OrdenMantenimiento

También hereda de OrdenReparacion mediante extends y representa un mantenimiento programado al que el taller le aplica un descuento. Agrega un atributo privado descuento (double), que es un porcentaje, y usa super() en su constructor. Sobrescribe con @Override el método calcularCostoFinal(), que resta el descuento al costo base (por ejemplo, un mantenimiento de $60.000 con descuento del 10% queda en $54.000), y sobrescribe mostrarInformacion() para indicar el tipo de orden.

Clase Taller

Es la clase que agrupa todo el sistema. Tiene un atributo nombreTaller (String) y un arreglo Cliente[] llamado clientes, con su contador correspondiente y una capacidad máxima definida en el constructor, por ejemplo 10 clientes. Su método agregarCliente(Cliente c) sigue la misma lógica de verificación de límite que las demás clases, y su método mostrarTodosLosClientes() recorre el arreglo completo e invoca el método mostrarInformacion() de cada cliente, que a su vez muestra sus bicicletas y las órdenes de cada bicicleta.

La relación entre Taller y Cliente es de agregación: el taller conoce y administra a sus clientes a través del arreglo, pero un cliente podría, en teoría, existir como concepto independiente del taller que lo atiende.

Aplicación de los Temas del Curso

Encapsulamiento

En todas las clases los atributos son privados y solo se leen o modifican con getters y setters públicos. Los arreglos y sus contadores tampoco se manipulan desde afuera: se usan métodos como agregarBicicleta, agregarOrden y agregarCliente, que revisan el límite antes de guardar. Las clases hijas tampoco tocan directamente los atributos del padre; usan getCosto() para calcular el costo final.

Herencia

OrdenReparacion es la clase padre, y OrdenUrgente y OrdenMantenimiento la extienden con extends. Las hijas reutilizan los atributos y el constructor del padre mediante super() y agregan cada una su propio atributo (recargo y descuento).

Polimorfismo

Las clases hijas sobrescriben con @Override el método calcularCostoFinal(). En la clase Main las variables son de tipo OrdenReparacion pero los objetos creados pueden ser de cualquiera de los tres tipos, y el método calcularTotal() de Bicicleta recorre el arreglo llamando calcularCostoFinal() sin necesidad de saber de qué tipo es cada orden: cada objeto responde según su clase real.

Ciclo de vida de los objetos

Cada objeto nace cuando se instancia con new y su constructor, que inicializa sus atributos y crea sus arreglos con la capacidad indicada. Luego se agrega a la estructura del taller (Taller guarda Clientes, Cliente guarda Bicicletas y Bicicleta guarda Órdenes) y permanece vivo mientras esté referenciado en esos arreglos. Durante la ejecución se consulta y se muestra su información, y deja de existir cuando el programa termina o cuando ya no hay ninguna referencia hacia él.

Resumen de la Estructura

En conjunto, el sistema queda organizado en una jerarquía de cuatro niveles: Taller agrupa Cliente (agregación), Cliente agrupa Bicicleta (composición) y Bicicleta agrupa OrdenReparacion (asociación). Además, existe una pequeña jerarquía de herencia en la que OrdenUrgente y OrdenMantenimiento extienden de OrdenReparacion. Cada nivel usa un arreglo nativo con su contador para controlar cuántos elementos contiene, y cada clase sabe mostrar su propia información y la de los objetos que tiene almacenados en su arreglo. Esta estructura permite practicar el manejo de arreglos de objetos, el paso de mensajes entre clases, la herencia y el polimorfismo sin necesitar conceptos más avanzados como interfaces.

Repositorio del Código

El código fuente completo del prototipo, con los siete archivos .java descritos en este documento (Taller, Cliente, Bicicleta, OrdenReparacion, OrdenUrgente, OrdenMantenimiento y Main), está disponible en el siguiente repositorio de GitHub: https://github.com/privatednicolas/RodaFix-Taller-Bicicletas-POO1